



Desarrollo de Aplicaciones Móviles

Tecnologías de Información II

Carlos Rojas Sánchez

Licenciatura en Informática

Universidad del Mar

¿Qué es una aplicación móvil?

- Software diseñado para ejecutarse en dispositivos móviles.
- Funciona en smartphones y tablets.
- Aprovecha sensores y capacidades del dispositivo.
- Puede ser nativa, híbrida o web.

Importancia de las aplicaciones móviles

- Gran crecimiento del uso de smartphones.
- Acceso rápido a servicios y contenido.
- Comunicación instantánea.
- Automatización de tareas cotidianas.
- Impacto en negocios y educación.

Historia del desarrollo móvil

- Primeros teléfonos con funciones básicas.
- Aparición de sistemas operativos móviles.
- Nacimiento de las tiendas de aplicaciones.
- Evolución hacia aplicaciones inteligentes.

Tipos de aplicaciones móviles

Aplicaciones Nativas

Desarrolladas específicamente para Android o iOS.

Aplicaciones Web

Funcionan desde el navegador móvil.

Aplicaciones Híbridas

Combinan tecnologías web y móviles.

Android

- Desarrollado por Google.
- Basado en Linux.
- Muy utilizado mundialmente.

iOS

- Desarrollado por Apple.
- Ecosistema cerrado.
- Alto rendimiento y seguridad.

Arquitectura básica de una app móvil

1. Interfaz de usuario (UI)
2. Lógica de negocio
3. Gestión de datos
4. Comunicación con servidores

Plataforma	Lenguajes
Android	Java, Kotlin
iOS	Swift, Objective-C
Híbridas	JavaScript, Dart

Frameworks populares

- Flutter
- React Native
- Ionic
- Xamarin

- Android Studio
- Xcode
- Visual Studio Code
- Git y GitHub

- Diseño responsivo.
- Experiencia de usuario (UX).
- Interfaz de usuario (UI).
- Navegación intuitiva.

Características de las apps móviles

- Uso de GPS.
- Acceso a cámara y micrófono.
- Notificaciones push.
- Funcionamiento offline.

- APIs REST.
- Intercambio de datos en JSON.
- Comunicación cliente-servidor.
- Servicios en la nube.

1. Planeación
2. Diseño
3. Desarrollo
4. Pruebas
5. Publicación
6. Mantenimiento

- Compatibilidad entre dispositivos.
- Seguridad de la información.
- Optimización de rendimiento.
- Consumo de batería.

Tendencias actuales

- Inteligencia Artificial móvil.
- Aplicaciones IoT.
- Realidad aumentada.
- Desarrollo multiplataforma.

- El desarrollo móvil es un área en crecimiento.
- Existen múltiples tecnologías y enfoques.
- Las aplicaciones móviles impactan diversos sectores.
- Es importante combinar diseño, programación y seguridad.

Línea de Tiempo y Datos de Uso en México

Evolución de los dispositivos móviles en México

- El uso de dispositivos móviles ha crecido rápidamente en las últimas décadas.
- México es uno de los países con mayor adopción de smartphones en América Latina.
- El acceso móvil ha transformado la comunicación, educación y comercio.
- Internet móvil supera actualmente al acceso desde computadoras en muchos hogares.

Década de 1990: Telefonía móvil básica

- Aparición de los primeros teléfonos celulares en México.
- Equipos grandes y costosos.
- Redes analógicas y posteriormente GSM.
- Uso principalmente empresarial.

Características

- Llamadas y mensajes SMS.
- Cobertura limitada.
- Alto costo por minuto.

2000 – 2007: Expansión de la telefonía móvil

- Reducción del costo de los dispositivos.
- Incremento del acceso a celulares prepago.
- Popularización de mensajes SMS.
- Aparición de teléfonos con cámara y reproducción multimedia.

Cambio importante

Los teléfonos móviles comenzaron a ser accesibles para gran parte de la población.

2007 – 2012: Llegada de los smartphones

- Introducción del iPhone y Android.
- Nacimiento de las tiendas de aplicaciones.
- Expansión del Internet móvil 3G.
- Incremento del uso de redes sociales móviles.

Aplicaciones populares

- Facebook
- WhatsApp
- YouTube

- Expansión de redes 4G LTE.
- Incremento del comercio electrónico móvil.
- Uso de aplicaciones bancarias y educativas.
- Crecimiento del desarrollo de apps nacionales.

Smartphones como principal medio de acceso a Internet

- Uso masivo de aplicaciones móviles.
- Expansión de pagos digitales.
- Uso de videollamadas y educación en línea.
- Inicio de implementación de redes 5G.

Impacto

Los dispositivos móviles se volvieron esenciales en actividades cotidianas.

Usuarios de teléfonos móviles en México

Año	Usuarios Aproximados
2000	14 millones
2005	47 millones
2010	81 millones
2015	104 millones
2020	122 millones
2025	Más de 130 millones

Fuente: estimaciones basadas en INEGI e IFT.

- Más del 80% de los usuarios acceden a Internet desde smartphones.
- Las principales actividades son:
 - Redes sociales
 - Mensajería instantánea
 - Streaming de video
 - Educación y trabajo remoto

Dato relevante

El teléfono móvil es el principal dispositivo de conexión en México.

Aplicaciones móviles más utilizadas

Comunicación

- WhatsApp
- Telegram
- Messenger

Entretenimiento

- YouTube
- TikTok
- Netflix

También destacan aplicaciones bancarias, educativas y de comercio electrónico.

- Reducción de la brecha digital.
- Expansión de cobertura 5G.
- Mayor seguridad y privacidad.
- Desarrollo de aplicaciones con IA.
- Incremento del Internet de las Cosas (IoT).

Perspectiva

El ecosistema móvil continuará creciendo en México durante los próximos años.

Arquitecturas y Patrones de Diseño de Apps

¿Qué es una arquitectura de software móvil?

- Es la estructura organizacional de una aplicación.
- Define cómo interactúan los componentes del sistema.
- Facilita mantenimiento y escalabilidad.
- Mejora reutilización y calidad del código.

Objetivo

Separar responsabilidades para desarrollar aplicaciones más organizadas.

Importancia de usar arquitecturas

- Facilita el trabajo en equipo.
- Reduce errores y duplicidad de código.
- Mejora pruebas y depuración.
- Permite escalar aplicaciones fácilmente.
- Favorece mantenimiento a largo plazo.

Arquitectura tradicional por capas

1. Capa de presentación
2. Capa lógica o negocio
3. Capa de datos

Ventaja

Permite separar claramente la interfaz, lógica y almacenamiento.

Modelo

- Maneja datos.
- Reglas de negocio.

Vista

- Interfaz gráfica.
- Presentación visual.

Controlador

- Gestiona eventos.
- Comunicación entre capas.

Uso

Muy utilizado en aplicaciones web y móviles tradicionales.

- MVP significa:
 - Model
 - View
 - Presenter
- El Presenter contiene la lógica de presentación.
- La Vista es más simple que en MVC.
- Facilita pruebas unitarias.

Ventaja

Mayor desacoplamiento entre interfaz y lógica.

- MVVM significa:
 - Model
 - View
 - ViewModel
- Muy utilizado en Android y Flutter.
- Usa enlace de datos (Data Binding).
- La interfaz se actualiza automáticamente.

Ventaja

Mejor separación entre lógica y presentación visual.

Arquitectura Clean Architecture

- Propuesta por Robert C. Martin.
- Basada en independencia de capas.
- El núcleo no depende de frameworks externos.

1. Entidades
2. Casos de uso
3. Interfaces
4. Frameworks

Objetivo

Construir aplicaciones mantenibles y escalables.

Patrones de diseño comunes

Patrón	Función
Singleton	Instancia única
Factory	Creación de objetos
Observer	Comunicación entre componentes
Repository	Acceso a datos
Dependency Injection	Gestión de dependencias

Arquitecturas modernas en desarrollo móvil

- Uso de componentes desacoplados.
- Arquitecturas reactivas.
- Integración con APIs y microservicios.
- Uso de contenedores y servicios en nube.
- Desarrollo multiplataforma.

Tecnologías relacionadas

Flutter, React Native, Kotlin Multiplatform.

- Las arquitecturas ayudan a organizar aplicaciones.
- Los patrones facilitan reutilización y mantenimiento.
- MVC, MVP y MVVM son ampliamente utilizados.
- Clean Architecture mejora escalabilidad.
- Elegir la arquitectura depende del proyecto.

Una buena arquitectura reduce problemas
futuros.

Desarrollo de apps para Android

- **Open Source:** Basado en el kernel de Linux (AOSP).
- **Cuota de mercado:** Domina aproximadamente el 70% del mercado global móvil.
- **Versatilidad:** Smartphones, tablets, TVs, autos y wearables.
- **Google Play Store:** Principal canal de distribución con millones de apps.

Kotlin (Recomendado)

- Moderno y conciso.
- Interoperable con Java.
- Seguro contra *NullPointerExceptions*.

Java

- El lenguaje histórico de Android.
- Gran cantidad de legacy code.
- Muy robusto y documentado.

- Basado en **IntelliJ IDEA**.
- **Build System:** Utiliza Gradle para gestionar dependencias.
- **Layout Editor:** Diseño visual de interfaces.
- **Logcat:** Herramienta de depuración en tiempo real.
- **Emulador:** Simulación de hardware de alto rendimiento.

Componentes de una Aplicación

Existen 4 pilares fundamentales en Android:

1. **Activities:** La interfaz de usuario (pantallas).
2. **Services:** Tareas en segundo plano.
3. **Broadcast Receivers:** Escuchan eventos del sistema.
4. **Content Providers:** Gestión y compartido de datos.

Ciclo de Vida (Activity Lifecycle)

Es crucial para gestionar la memoria y la experiencia de usuario:

- `onCreate()`: Inicialización.
- `onStart()` / `onResume()`: La app se vuelve visible y activa.
- `onPause()` / `onStop()`: La app pasa a segundo plano.
- `onDestroy()`: Liberación de recursos.

- Separación de lógica (Kotlin/Java) y diseño (XML).
- **ViewGroups:** `LinearLayout`, `ConstraintLayout`, `FrameLayout`.
- **Widgets:** `Button`, `TextView`, `ImageView`.
- Uso de recursos en `res/layout/`.

- El nuevo paradigma de UI **declarativa**.
- Escrito totalmente en Kotlin.
- Elimina la necesidad de archivos XML.
- Reutilización de componentes mediante funciones **@Composable**.
- Actualización automática de la UI según el estado.

Conjunto de librerías para seguir las mejores prácticas:

- **Navigation:** Gestión de flujos entre pantallas.
- **Room:** Abstracción sobre bases de datos SQLite.
- **WorkManager:** Tareas programadas garantizadas.
- **ViewModel:** Persistencia de datos ante cambios de configuración.

- **Retrofit:** La librería estándar para peticiones HTTP/REST.
- **Serialization:** Kotlin Serialization o GSON para JSON.
- **Glide / Coil:** Librerías para carga eficiente de imágenes.
- **Corrutinas:** Manejo de hilos asíncronos sin bloquear la UI.

Model-View-ViewModel es el estándar recomendado:

- **View:** Observa el estado del ViewModel.
- **ViewModel:** Contiene la lógica de negocio y prepara datos.
- **Model:** Fuente de verdad (API o Base de datos).
- Beneficio: Facilita enormemente las pruebas unitarias.

- Android sigue las líneas de **Material Design 3**.
- **Colores Dinámicos:** La app se adapta al fondo de pantalla del usuario.
- **Accesibilidad:** Enfoque en contraste y tamaños de fuente.
- **Interactividad:** Animaciones fluidas y transiciones.

- **AndroidManifest.xml:** Declaración obligatoria de intenciones.
- **Permisos en tiempo de ejecución:** Desde Android 6.0 (Cámara, GPS, Contactos).
- **Scoped Storage:** Limitación de acceso a archivos para privacidad.

Publicación: Android App Bundle (.aab)

- Sustituye al antiguo APK.
- **Optimización:** Google Play genera un APK específico para cada dispositivo.
- **Tamaño reducido:** Menor peso de descarga para el usuario.
- Firma de la app gestionada por Google.

- **Unit Tests:** (JUnit) Lógica de negocio pura (JVM local).
- **Instrumented Tests:** Se ejecutan en un dispositivo real.
- **Espresso:** Automatización de UI y flujos de usuario.
- **Compose Test:** Herramientas específicas para probar UI declarativa.

- **Multiplataforma:** Kotlin Multiplatform (KMP).
- **IA:** Integración de Gemini Nano en dispositivos.
- **Plegables:** Adaptación de interfaces a nuevos form-factors.
- **Continuidad:** Mejor integración con ChromeOS y Windows.

Desarrollo de apps para iOS

- **Cerrado y Controlado:** Integración vertical entre hardware y software.
- **Calidad sobre Cantidad:** Estándares de diseño y rendimiento muy estrictos.
- **Rentabilidad:** Mayor gasto promedio por usuario en la App Store.
- **Seguridad:** Enfoque en la privacidad (App Tracking Transparency).

- Lanzado por Apple en 2014 para reemplazar a Objective-C.
- **Seguro:** Manejo estricto de tipos y *optionals* para evitar crashes.
- **Rápido:** Rendimiento cercano a C++.
- **Sintaxis:** Limpia, moderna y fácil de leer.
- **Open Source:** Aunque mantenido por Apple, es de código abierto.

- Disponible únicamente en **macOS**.
- **Interface Builder:** Diseño visual (Storyboards/XIBs).
- **Swift Playgrounds:** Prototipado rápido de código.
- **Simuladores:** Pruebas en todos los modelos de iPhone y iPad.
- **Instruments:** Herramienta avanzada para medir rendimiento y fugas de memoria.

Componentes Esenciales

- **UIKit:** El framework clásico basado en imperatividad.
- **SwiftUI:** El nuevo estándar declarativo.
- **Foundation:** Librerías base (fechas, datos, redes).
- **Cocoa Touch:** Capa de abstracción para interfaces táctiles.

Ciclo de Vida (App Lifecycle)

Gestionado por el `SceneDelegate` o `AppDelegate`:

- **Active:** La app está en primer plano y recibe eventos.
- **Inactive:** Estado de transición (ej. entra una llamada).
- **Background:** Ejecución limitada en segundo plano.
- **Suspended:** La app está en memoria pero no ejecuta código.

- Introducido en 2019.
- **Declarativo:** Defines "qué" quieres en la pantalla, no "cómo" dibujarlo.
- **Previews en vivo:** Cambios en tiempo real sin recompilar.
- **Multiplataforma:** El mismo código puede funcionar en iPhone, Mac y Apple Watch.

- **Swift Package Manager (SPM):** La solución nativa e integrada en Xcode.
- **CocoaPods:** El gestor más veterano basado en Ruby.
- **Carthage:** Alternativa descentralizada que prioriza la velocidad de compilación.

- **UserDefaults:** Para configuraciones pequeñas (flags, preferencias).
- **SwiftData / Core Data:** Frameworks potentes para bases de datos relacionales y grafos de objetos.
- **File System:** Acceso directo al sandbox de la aplicación.
- **Keychain:** Almacenamiento seguro de contraseñas y tokens.

- Introducido para simplificar el código asíncrono.
- **Structured Concurrency:** Evita condiciones de carrera (*race conditions*).
- **Actors:** Protegen el estado mutable para que sea seguro entre hilos.
- Sustituye al antiguo *Grand Central Dispatch* (GCD).

- **Claridad:** Texto legible y botones precisos.
- **Deferencia:** El contenido debe ser el protagonista.
- **Profundidad:** Capas visuales y desenfokes (Glassmorphism).
- **SF Symbols:** Miles de iconos consistentes integrados en el sistema.

- **URLSession:** API nativa para tareas de red.
- **Codable:** Protocolo para convertir JSON a objetos Swift de forma automática.
- **Alamofire:** Popular librería externa que simplifica la sintaxis de red.

- **MVVM:** El estándar para SwiftUI.
- **Clean Architecture:** Separación estricta de capas para apps escalables.
- **The Composable Architecture (TCA):** Un enfoque funcional y unidireccional muy popular hoy.

- **ARKit:** Realidad aumentada de alto nivel.
- **Core ML:** Integración de modelos de Machine Learning.
- **HealthKit:** Acceso a datos de salud y fitness.
- **HomeKit:** Control de domótica desde el iPhone.

- **XCTest:** Framework nativo para Unit y UI Tests.
- **TestFlight:** Plataforma oficial para distribución de betas a usuarios externos.
- **XCUITest:** Pruebas funcionales simulando toques en la pantalla.

- Proceso de revisión manual por parte de Apple.
- **App Store Optimization (ASO):** Palabras clave y capturas de pantalla.
- **Directrices de Revisión:** Reglas estrictas contra spam y contenido inseguro.
- El pago anual de la cuenta de desarrollador (\$99 USD).

Desarrollo de apps para HarmonyOS

- **HMS (Huawei Mobile Services):** La alternativa propia a los GMS de Google.
- **HarmonyOS:** Sistema operativo distribuido para múltiples dispositivos.
- **AppGallery:** La tercera tienda de aplicaciones más grande del mundo.
- **Estrategia 1+8+N:** Conectividad total entre smartphone, tablets y hogar.

- **Compatibilidad:** Soporta Java y Kotlin para apps de Android (AOSP). Para el ecosistema nativo de HarmonyOS:
- **ArkTS:** Basado en TypeScript, optimizado para alto rendimiento.
- **C/C++:** Para módulos de bajo nivel y motores de juegos.

- Basado en el motor de **IntelliJ IDEA**.
- Diseñado específicamente para el desarrollo de **HarmonyOS**.
- **Low-code development:** Soporta diseño mediante arrastrar y soltar.
- **Multi-device Preview:** Visualización simultánea en TV, móvil y reloj.

Huawei ofrece reemplazos directos para funcionalidades de Google:

- **Account Kit:** Autenticación segura de usuarios.
- **Push Kit:** Notificaciones en tiempo real.
- **Map Kit & Location Kit:** Geolocalización y mapas propios.
- **Ads Kit:** Monetización mediante anuncios.

- **Omni-resiliente:** Diseñado para funcionar en dispositivos con 128KB o gigas de RAM.
- **Soft Bus Distribuido:** Permite que varios dispositivos actúen como uno solo.
- **Virtualización de Hardware:** Usar la cámara del teléfono desde una tablet sin cables.

El Concepto de "Ability"

En lugar de Activities, HarmonyOS usa **Abilities**:

- **FA (Feature Ability)**: Con interfaz de usuario (similar a la Activity).
- **PA (Particle Ability)**: Tareas de servicio o datos (sin interfaz).
- Permiten la migración de tareas entre dispositivos en caliente.

- Framework declarativo similar a SwiftUI o Jetpack Compose.
- **Eficiencia:** Reduce las líneas de código hasta en un 40%.
- **Componentes:** Botones, listas y grids optimizados para pantallas táctiles.

Para apps que ya existen en Android:

- **Convertor:** Analiza el código y detecta dependencias de Google (GMS).
- **Auto-reemplazo:** Sustituye automáticamente clases de Google por las de Huawei.
- Genera un reporte de compatibilidad detallado.

- Motor de búsqueda propio integrado en el sistema.
- Indexa directamente las aplicaciones de la AppGallery.
- Permite "Quick Apps" (apps que no requieren instalación).

- **KV Store:** Almacenamiento de clave-valor distribuido.
- **Relational Database:** Basado en SQLite con capas de seguridad.
- **Cloud Storage:** Sincronización con la nube de Huawei.

- **Microkernel:** Mayor seguridad estructural que kernels monolíticos.
- **TEE (Trusted Execution Environment):** Aislamiento de datos biométricos.
- Certificaciones de seguridad de alto nivel internacional.

- **Merchant Service:** Gestión de pagos globales.
- **App Bundles:** Huawei también soporta distribución optimizada por dispositivo.
- **A/B Testing:** Pruebas nativas dentro de la tienda.

Como el hardware físico es variado:

- Acceso remoto a dispositivos reales en los data centers de Huawei.
- **Cloud Testing:** Pruebas automatizadas de compatibilidad y estabilidad.
- No requiere tener el teléfono físico para depurar.

- **IAP (In-App Purchases):** Gestión de suscripciones y productos consumibles.
- **Wallet Kit:** Integración con pagos móviles y tarjetas de fidelidad.
- Soporte para monedas locales en más de 170 regiones.

- **Independencia:** Reducción total de dependencia de tecnologías externas.
- **OpenHarmony:** La versión open source para la comunidad.
- **Oportunidad:** Un mercado masivo en China y creciente en mercados emergentes.